

Resolución y Pauta de Corrección: Optimización de Carga Logística

Unidad: Algoritmos Devoradores (Greedy) • Mochila Fraccionaria vs. Mochila 0/1

Parte 1. Solución Algorítmica (Java / Spring Boot Context)

A continuación se presenta la implementación de referencia del algoritmo devorador para el **Problema de la Mochila Fraccionaria** con complejidad $O(n \log n)$:

```
import java.util.Arrays;
import java.util.Comparator;

public class MochilaFraccionaria {

    public static class Item {
        String nombre;
        double peso;
        double valor;

        public Item(String nombre, double peso, double valor) {
            this.nombre = nombre;
            this.peso = peso;
            this.valor = valor;
        }

        public double getDensidad() {
            return valor / peso;
        }
    }

    public static double resolverMochilaFraccionaria(Item[] items, double capacidad) {
        // 1. Ordenar por densidad v_i / p_i desc. O(n log n)
        Arrays.sort(items, Comparator.comparingDouble(Item::getDensidad).reversed());

        double beneficioTotal = 0.0;
        double pesoActual = 0.0;

        // 2. Selección codiciosa/devoradora. O(n)
        for (Item item : items) {
            if (pesoActual + item.peso <= capacidad) {
                pesoActual += item.peso;
                beneficioTotal += item.valor;
                System.out.println("Se toma 100% de " + item.nombre);
            } else {
                double remanente = capacidad - pesoActual;
                double fraccion = remanente / item.peso;
                beneficioTotal += item.valor * fraccion;
                pesoActual += remanente;
                System.out.printf("Se toma %.2f%% de %s\n", fraccion * 100, item.nombre);
                break; // Capacidad colmada
            }
        }
        return beneficioTotal;
    }
}
```

```
}  
}
```

Parte 2. Respuestas Teóricas y Demostraciones

1. Criterio de Selección e Impacto de Criterios Alternativos

Demostración de optimalidad del criterio v_i / p_i : Al permitir fraccionar los objetos, la estrategia devoradora busca maximizar el "valor por unidad de capacidad consumida". Cada unidad de peso que ocupa un objeto con mayor densidad aporta el máximo beneficio posible al espacio disponible en la mochila.

Contraejemplos de criterios subóptimos:

- **Mayor Valor (v_i):** Capacidad $P = 10$.
 - Item A: $p_1 = 10, v_1 = 100$ ($v/p = 10$).
 - Item B: $p_2 = 5, v_2 = 90$ ($v/p = 18$). Item C: $p_3 = 5, v_3 = 90$ ($v/p = 18$).
 - *Greedy por Valor elige A:* Beneficio = 100.
 - *Óptimo (por densidad B+C):* Beneficio = 180.
- **Menor Peso (p_i):** Capacidad $P = 10$.
 - Item A: $p_1 = 1, v_1 = 2$ ($v/p = 2$).
 - Item B: $p_2 = 10, v_2 = 100$ ($v/p = 10$).
 - *Greedy por Menor Peso elige A completando con B (90%):* Beneficio = $2 + 90 = 92$.
 - *Óptimo (por densidad B al 100%):* Beneficio = 100.

2. Complejidad Algorítmica con Entrada Preordenada

Si el arreglo de objetos se recibe preordenado por densidad decreciente, el costo del ordenamiento $O(n \log n)$ se elimina. El bucle de selección devorador recorre la lista como máximo una vez, logrando una complejidad temporal final de $O(n)$ (tiempo lineal) y complejidad espacial $O(1)$ auxiliar.

Parte 3. Resolución del Desafío Integrador (Mochila 0/1)

Dado el conjunto de datos con capacidad $P = 10$:

Objeto	Peso (p_i)	Valor (v_i)	Relación (v_i / p_i)
Objeto A	7	70	10 (Mayor)
Objeto B	5	45	9
Objeto C	5	45	9

Resultado de la Estrategia Devoradora (0/1):

1. Selecciona **Objeto A** ($p = 7, v = 70$) por tener la mayor densidad (10).
2. Capacidad restante: $10 - 7 = 3$.
3. Intenta evaluar B y C ($p = 5$ cada uno). Ninguno entra completo ($x_B = 0, x_C = 0$).

4. **Resultado Devorador 0/1:** Selección = {A}. Beneficio Total = **70**. Capacidad sobrante = 3.

Análisis de Optimalidad y Reflexión para la Clase:

El **Óptimo Global** para la variante 0/1 es seleccionar {B, C}, con peso combinado $5 + 5 = 10$ y Beneficio Total = $45 + 45 = 90$.

Conclusión Teórica: La estrategia devoradora **falla en el problema 0/1** porque al tomar el objeto A de mayor densidad, deja un hueco vacante de peso 3 incapaz de albergar otros elementos. Al no poder fraccionar A, esa capacidad sobrante se desperdicia, perdiendo la optimalidad global. (Este problema requiere Programación Dinámica).